CBPL-kit: Technical documentation

Implementation and Algorithm Details

Seong Lee, Junsang Park, and Youn-Woo Lee

Last updated: October 19, 2026

https://cbpl-kit.vercel.app

1. Browser runtime

At startup, CBPL-kit loads NumPy, SciPy, and PyYAML through `pyodide.loadPackage`, then installs Plotly with `micropip`, enabling numerical computation, YAML parsing, and figure generation directly in the browser. On the initial page load, the browser downloads approximately 30 MB of runtime and package files. Once initialized, CBPL-kit performs all calculations locally, without sending computation requests to a server.

2. Package structure

The computational logic is organized as a modular Python package (Table 1), available in a public GitHub repository (https://github.com/justinp54/CBPL-kit). The core computation classes carry no dependency on the plotting layer (Plotly) and can therefore be utilized independently outside the browser, for example in Jupyter notebooks or automated tests. The numerical modules are covered by an automated test suite (pytest), and every change is checked in continuous integration by linting, the test suite, and a consistency check between the package and the copy of it served to the browser. For readers who wish to adapt the code itself, the repository also includes an AGENTS.md guide written for AI-assisted development tools.

3. Numerical methods

3.1. Coordinate Convention

The three components are numbered 1 for the carrier, 2 for the solute, and 3 for the solvent, and w_i denotes the mass fraction of component i. All geometric calculations are performed after mapping the ternary mass fractions to Cartesian coordinates on an equilateral triangle using Eq. (1), with the solvent, carrier, and solute vertices at lower-left, lower-right, and top of the triangle throughout this work. Under this convention, the auxiliary directions used in diagonal tie-line interpolation and extrapolation through the conjugate curve have slopes $+\sqrt{3}$ and $-\sqrt{3}$, parallel to two sides of the triangle.

$$x = w_1 + \frac{1}{2}w_2, \qquad y = \frac{\sqrt{3}}{2}w_2 \tag{1}$$

3.2. Binodal Curve Construction

The measured equilibrium compositions provided by the System tab were mapped to Cartesian coordinates and sorted along the binodal curve. When the measurements do not extend to the edges where solute concentration is zero, anchor points were added to supply the missing intercepts. Their locations were estimated from the outermost endpoints of the measurements using the shape-preserving Fritsch-Carlson method.[1] These synthesized anchor points define the limits of the interpolation domain and prevent nonphysical fluctuation of the fitted binodal curve near the binary phase edges.

The geometric apex of the binodal is refined by a local quadratic polynomial through the highest measured point and its two neighbors. The vertex of this polynomial serves as the criterion for dividing the data into carrier-rich and solvent-rich branches. The vertex is accepted only if it falls between the two neighbouring points and rises no more than about 2 wt% solute above the highest measured point; otherwise the measured maximum is used unchanged. Each branch is then represented independently by a monotone piecewise-cubic Hermite interpolant, using the same shape-preserving construction. The first derivatives of both branches were constrained to zero at the apex so that they can join smoothly with a horizontal tangent. The geometric apex referenced here is used only to construct two single-valued interpolation branches and was not identified with the plait point.

Two independent monotone interpolants are used instead of a single global cubic spline because a global spline enforces curvature continuity across all knots and can therefore oscillate or overshoot when the equilibrium points are unevenly spaced. By contrast, the branchwise shape-preserving interpolants used here stay within the local range of their input data, so they neither overshoot nor fluctuate in ways the measurements do not support.

3.3. Composition Recovery

The full ternary composition of each tie-line endpoint is recovered from the solute mass fraction supplied as input. Because each endpoint is constrained to lie on its corresponding side of the binodal curve, the input mass fraction specifies a unique point on that branch. This point is located by a bounded search on the appropriate branch using Brent's method, and the remaining solvent and carrier fractions follow from its coordinates.

Product-stream compositions are recovered in a similar manner from the solute content of each stream, which may be supplied in one of three forms: a titration volume, a molar concentration, or a mass fraction. A titration volume is first converted to a molar concentration as $C = C_{\mathrm{NaOH}} V_{\mathrm{NaOH}} f / V_{\mathrm{aliquot}}$, where f is the dilution factor, assuming 1:1 acid–base neutralization. For a molar concentration, the corresponding concentration at any point on the appropriate binodal branch is computed under the volume-additivity assumption, and the stream composition is obtained by minimizing the absolute deviation

[1] F. N. Fritsch and R. E. Carlson, Monotone piecewise cubic interpolation, *SIAM Journal on Numerical Analysis* **17**(2), 238–246 (1980).

using Brent's bounded minimization algorithm (SciPy `minimize_scalar` with tolerance 10^{-5}). For a mass fraction, the stream point is located on the branch directly by the same bounded root search used for the tie-line endpoints. The three forms meet in a single function (`solute_concentration` in `config.py`).

3.4. Conjugate Curve and Plait Point

Following Treybal's construction,[2] auxiliary lines of slope $-\sqrt{3}$ and $+\sqrt{3}$ were drawn from the solvent-rich and carrier-rich endpoints of each tie line; their intersection defines one point of the conjugate curve. The two x-intercepts of the binodal curve are treated as a degenerate tie line and supply the first such point, anchoring the low-solute end of the conjugate curve. In the horizontal variant, one of the two auxiliary lines is replaced by the horizontal line parallel to the base of the triangle. The side to be replaced is selected automatically: both variants are constructed, and the one whose auxiliary points show the smaller total excursion into negative composition is kept. This selection guards against geometrically unreasonable conjugate points near the triangle boundary.

Near the plait region, the conjugate curve may reverse its direction in Cartesian x, making direct interpolation of the composition on the Cartesian coordinate unreliable. The points are therefore ordered along the curve and assigned a monotonically increasing parameter t, defined as the cumulative chordal distance between adjacent points, and the conjugate curve is interpolated parametrically as $x(t)$ and $y(t)$ with PCHIP interpolation method. For the plait-point estimate, the curve is extended past its last point along its exact tangent direction, which prevents the extension from bending back on itself. The point at which this extension comes closest to the equilibrium curve is taken as the estimate. An independent plait point estimate is obtained in Hand coordinates, $X_H = \ln(w_2/w_1)$, $Y_H = \ln(w_2/w_3)$: ordinary least-squares regression of the tie-line data gives the Hand correlation line, and its intersection with the transformed binodal curve is taken as the estimate of the plait point. It serves as an independent check on the geometric estimate.

3.5. Hunter-Nash Stage Construction

Ideal stages were generated by alternating tie line and operating line constructions. Starting from the final extract's composition E_1, an auxiliary line of slope $-\sqrt{3}$ is intersected with the conjugate curve, and a line of slope $+\sqrt{3}$ from that intersection back to the carrier-rich branch of the binodal curve gives the composition of the corresponding raffinate of that stage. The operating line through the operating point P then gives the next extract, and the alternation repeats. The stepping procedure terminates when the raffinate reaches the target composition R_N, and the last stage is obtained by linear interpolation between the final two raffinate compositions, $N = (n-1) + (y_{n-1} - y_{R_N})/(y_{n-1} - y_n)$, where n indexes the stepped stages. The solver also distinguishes convergence from a

[2] R. E. Treybal, *Mass-Transfer Operations*, 3rd ed., McGraw-Hill, New York (1980).

stalled construction, in which successive extract points stop moving before the target is reached.

3.6. Lever Rule

The mixture point M is the intersection of the E_1–R_N and E_{N+1}–R_0 lines, and the operating point P is obtained as the intersection of the extensions of the R_0–E_1 and R_N–E_{N+1} lines. As a mass balance check, an independent mixture point M' is calculated directly from the measured feed and solvent mass flow rates by Eq. (2).

$$w_{M'} = \frac{\dot{m}_F w_F + \dot{m}_S w_S}{\dot{m}_F + \dot{m}_S} \tag{2}$$

where $\dot{m}_F$ and $\dot{m}_S$ denote the feed and solvent mass flow rates and w_F and w_S their compositions. The first-stage extract composition E_1' is then obtained by intersecting the extension of the R_N–M' line with the solvent-rich branch of the binodal curve. The agreement between M and M', and between E_1 and E_1' provides an independent check of the Hunter-Nash construction.

4. Feasible Explorer range searches

The S:F Explorer bounds its slider by two searches, $(\mathrm{S/F})_{\min}$ and $(\mathrm{S/F})_{\max}$. The upper limit corresponds to the mixture point M reaching the binodal curve itself; beyond that ratio the mixture is a single phase and the separation is impossible. The lower limit is found from the relation between the operating line and tie line. As the ratio decreases, the operating point P moves along the fixed R_N–E_{N+1} line. The limit is reached when an operating line through P coincides with a tie line – equivalently, when P itself lies on the extension of one of the measured tie lines. At that ratio the driving force vanishes, and no finite number of stages can reach the final extract composition. When a system's measured tie lines never produce such a crossing, no limit is reported and the slider falls back to a fixed span.

Unlike the solvent ratio, the feed composition has no general analytical feasibility limit, and the mode of construction failure differs from system to system. At the fixed experimental flow rates, a feed composition is therefore considered feasible if it produces a valid mixture point, a corresponding E_1' on the binodal, and a finite operating point P. Starting from the experimental feed composition, the feed solute mass fraction is explored in both directions until the construction first becomes infeasible, and each resulting boundary is then refined by bisection.

The stage-count trend charts are sampled from the safe end of each span toward its limiting end. Isolated failures near the single-phase end, which arise from edge cases, are skipped, and only the points that could be computed are plotted.

5. Data validation

Validation of the data operates at two different levels: structural checks before a system is built, and numerical guards once it is in use. Before a system is built, the entries themselves are checked for missing or non-numeric values, and equilibrium rows not sorted by increasing order of carrier-fraction are flagged, with one-click reordering offered. Each composition row is also required to sum to 100% within 0.5%, and a system must supply at least three equilibrium rows and two tie lines. The checks are implemented in a single module fashion, in `validate_system.py` (Table 1), shared by the System-tab form and the automated workflow that validates a system contributed through the project repository. Once the data are accepted, a second level guards the computation, the points are re-sorted in the diagram coordinate and near-duplicate points that would degenerate the binodal construction are rejected with a message naming the offending rows. Any error raised during a later calculation is reported in the interface rather than silently producing a distorted diagram.

Table 1: Modules of the CBPL-kit Python package.

Module	Class/function	Responsibility
`config.py`	`constants`	Loads the binodal curve and tie-line data, densities, and molar masses from the YAML file, and exports the experimental conditions
`ternary.py`	`xy_to_comp`, `comp_to_xy`	Transforms between ternary mass fraction triplets and Cartesian coordinates using the equilateral-triangle projection
`equilibrium.py`	`EquilibriumSystem`	Constructs the binodal curve, resolves tie-line endpoints on it, and places a product stream on its branch
`conjugate.py`	`ConjugateCurve`	Constructs the conjugate curve and locates the plait point
`correlation.py`	correlation fits	Fit the Othmer-Tobias, Hand, and Bachman correlations, compute the distribution coefficients and the selectivity, and build the Treybal log-log plait point estimation
`hunter_nash.py`	`HunterNashSolver`	Performs the Hunter-Nash stepping construction and reports the fractional stage count
`lever_rule.py`	solver functions	Computes lever-rule constructions and feasibility searches for two explorers
`plot_util.py`	plot functions	Generates the figure specifications rendered by Plotly
`validate_system.py`	`validate`	Structural validation of the system definitions, shared by the System-tab form and the automated submission workflow
`main.py`	`main`	Runs the whole analysis outside the browser, from the solute concentration and flow rates to the stream compositions, the stage construction, and the lever-rule check